

TUGAS

PEMOGRAMAN BERORIENTASI OBJEK

Dosen Pengampu:
SIROJ NUR ULUM, S.T., M.T.



Disusun Oleh:
MUHAMAD FAJRI
233040105

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS PASUNDAN
BANDUNG
2025

Bab 9: Building Objects and Object-Oriented Design

Bab 9 dalam buku *The Object-Oriented Thought Process* oleh Matt Weisfeld membahas konsep **komposisi**, yang merupakan cara mendesain kelas dalam pemrograman berorientasi objek (OOP) melalui hubungan antara objek yang berbeda. Konsep utama yang dibahas berfokus pada **komposisi, agregasi, asosiasi, dan kardinalitas**.

1. Komposisi

Komposisi adalah hubungan di mana suatu objek terdiri dari objek lain sebagai bagian integralnya. Hubungan ini sering disebut "**has-a**" relationship. Jika objek utama dihapus, objek yang dikandungnya juga ikut dihapus.

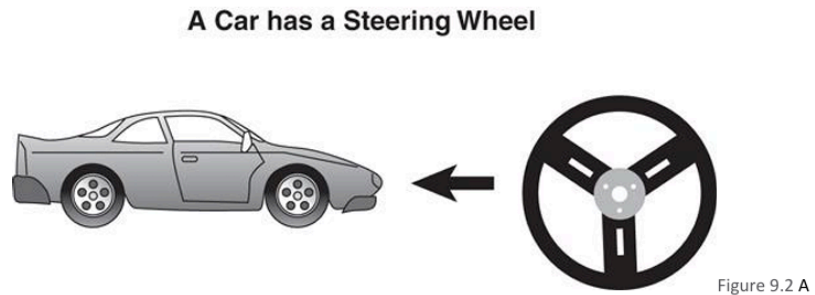
Komposisi sangat penting dalam desain perangkat lunak yang modular dan reusable. Dengan menggunakan komposisi, kita dapat membuat sistem yang lebih fleksibel dan mudah diperluas tanpa harus mengubah kode yang sudah ada. Salah satu keuntungan utama dari komposisi adalah meningkatkan **enkapsulasi**, karena objek bagian tidak dapat diakses langsung oleh objek luar.

Komposisi digunakan untuk membangun sistem dengan menggabungkan bagian-bagian yang lebih sederhana menjadi satu kesatuan yang lebih kompleks. Ini merupakan cara alami bagi manusia untuk menyelesaikan masalah, karena membantu kita mengorganisir konsep-konsep dalam unit yang lebih mudah dipahami. Misalnya, kita tidak menyebutkan satu per satu bahwa sebuah mobil memiliki roda, setir, mesin, dan pintu. Sebaliknya, kita hanya mengatakan bahwa itu adalah "mobil".

Contoh:

- **Mobil memiliki setir**

Jika mobil dihapus, maka setirnya juga ikut hilang.



Dalam UML, komposisi direpresentasikan dengan garis yang diakhiri dengan **belah ketupat hitam**.

2. Agregasi

Agregasi adalah bentuk komposisi yang lebih intuitif, dimana suatu objek kompleks terdiri dari objek-objek lain. Ketika melihat suatu objek yang kompleks, kita sering melihatnya sebagai satu kesatuan tanpa menyadari bahwa objek tersebut terdiri dari banyak bagian kecil

Sebagai contoh, sebuah **TV** tampak sebagai satu unit yang berfungsi sebagai perangkat hiburan. Namun, di dalamnya terdapat berbagai komponen seperti microchip, layar, tuner, dan lain-lain. Ketika seseorang pergi ke toko elektronik untuk membeli TV, mereka tidak membeli microchip, layar, dan tuner secara terpisah, tetapi membeli TV sebagai satu kesatuan.

Hal yang sama berlaku untuk **mobil**. Saat membeli mobil, seseorang tidak memilih setiap komponen seperti busi atau pegangan pintu secara terpisah, tetapi membeli mobil sebagai satu objek yang sudah memiliki semua bagian tersebut.

Dalam UML, agregasi direpresentasikan dengan garis yang diakhiri dengan **belah ketupat putih**, yang menunjukkan hubungan yang lebih fleksibel dibandingkan komposisi.

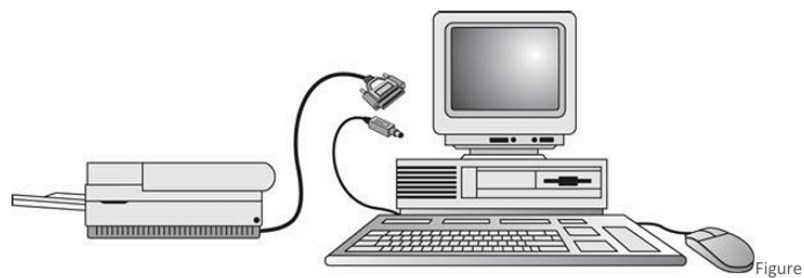
3. Asosiasi

Asosiasi adalah hubungan antar objek yang tidak selalu mencerminkan hubungan kepemilikan. Ini bisa berupa hubungan satu arah atau dua arah.

Asosiasi terjadi ketika satu objek meminta layanan dari objek lain. Hubungan ini lebih fleksibel dibandingkan agregasi dan komposisi.

Contoh:

- **sistem komputer desktop**



yang terdiri dari unit utama, monitor, keyboard, dan mouse. Meskipun setiap bagian merupakan objek yang terpisah, mereka saling berinteraksi melalui koneksi seperti kabel atau port USB.

4. Perbedaan Agregasi dan Asosiasi

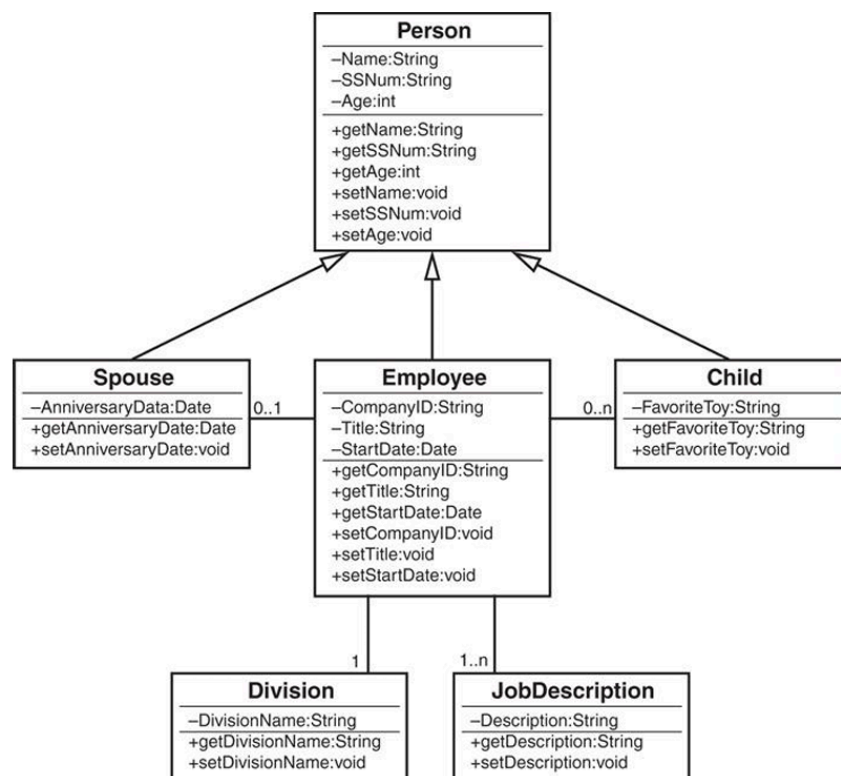
Perbedaan utama antara **agregasi** dan **asosiasi** adalah bahwa dalam agregasi, kita hanya melihat keseluruhan sebagai satu unit (misalnya, komputer sebagai satu perangkat). Sedangkan dalam asosiasi, setiap bagian tetap terlihat sebagai objek terpisah yang bekerja sama untuk menyediakan suatu layanan.

5. Kardinalitas

Kardinalitas menggambarkan **berapa banyak** objek yang dapat terhubung dalam suatu relasi. Dalam UML, ini direpresentasikan dengan angka seperti:

- **1** (satu)
- **1..1** (satu ke satu)
- **1..n** (satu ke banyak)
- **0..n** (bisa nol atau lebih banyak)

Kardinalitas sangat penting dalam desain basis data dan struktur objek karena menentukan hubungan antara entitas yang berbeda dalam sistem. Dan berikut adalah contoh diagram UML dengan kardinalitas:



6. Asosiasi dengan Banyak Objek (Multiple Object Associations)

Dalam banyak kasus, suatu objek dapat memiliki hubungan dengan lebih dari satu objek lainnya. Dalam kode, hubungan ini sering direpresentasikan dengan array atau koleksi lainnya.

Contoh dari buku: Pada kelas `Employee`, terdapat beberapa atribut yang memiliki hubungan **satu-ke-banyak**:

```
private Child[] child;  
private JobDescription[] jobDescriptions;
```

Atribut `child` dan `jobDescriptions` adalah contoh hubungan **satu-ke-banyak**, di mana seorang karyawan dapat memiliki beberapa anak dan beberapa deskripsi pekerjaan.

Bab 10: Design Patterns (Pola Desain)

Bab 10 dalam buku *The Object-Oriented Thought Process* oleh Matt Weisfeld membahas konsep **Pola Desain** dalam pemrograman berorientasi objek. Pola desain adalah solusi umum untuk masalah yang sering muncul dalam desain perangkat lunak. Pola ini memungkinkan pengembang untuk menerapkan praktik terbaik dalam membangun sistem yang modular, dapat digunakan kembali, dan mudah diperluas.

1. Konsep Dasar Pola Desain

Pola desain muncul karena pengembang perangkat lunak sering kali memodelkan sistem dunia nyata ke dalam kode. Sama seperti pola yang digunakan dalam rekayasa dan arsitektur, pola desain membantu menciptakan solusi yang lebih efisien dan dapat diterapkan kembali.

Salah satu referensi utama dalam pola desain adalah buku *Design Patterns: Elements of Reusable Object-Oriented Software* yang ditulis oleh **Gang of Four (GoF)**. Buku ini mendokumentasikan berbagai pola desain yang telah terbukti efektif dalam pengembangan perangkat lunak.

Pola desain terdiri dari empat elemen utama:

- **Nama pola**, Digunakan untuk mendeskripsikan pola desain secara ringkas.
- **Masalah yang diselesaikan**, Menjelaskan kapan pola ini diterapkan.
- **Solusi**, Menyediakan gambaran umum tentang bagaimana pola ini menyelesaikan masalah.
- **Konsekuensi**, Menggambarkan hasil serta kelebihan dan kekurangan dari penggunaan pola ini.

2. Kategori Pola Desain

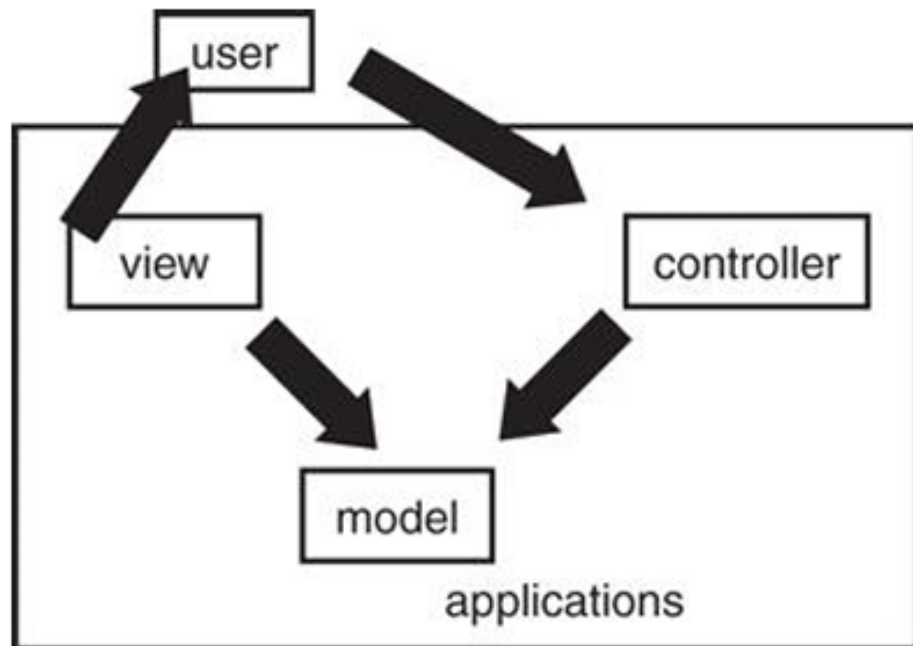
Pola desain dikategorikan ke dalam tiga kelompok utama:

- **Pola Kreasi (Creational Patterns)** – Berfokus pada cara menciptakan objek dengan cara yang lebih fleksibel dan efisien.
- **Pola Struktural (Structural Patterns)** – Menangani bagaimana kelas dan objek digabungkan untuk membentuk struktur yang lebih besar.
- **Pola Perilaku (Behavioral Patterns)** – Berfokus pada komunikasi antara objek dan bagaimana mereka bekerja sama.

3. Model-View-Controller (MVC) dalam Smalltalk

Salah satu contoh awal penerapan pola desain adalah **Model-View-Controller (MVC)**, yang digunakan dalam bahasa pemrograman **Smalltalk** untuk membangun antarmuka pengguna (UI). MVC membagi aplikasi menjadi tiga bagian utama:

- **Model** – Merepresentasikan data dan logika bisnis.
- **View** – Menampilkan data kepada pengguna.
- **Controller** – Mengatur interaksi antara pengguna dan sistem.



Dengan memisahkan antarmuka pengguna dari logika bisnis dan data, MVC membuat sistem lebih fleksibel dan mudah diubah tanpa mempengaruhi komponen lain.

Contoh penerapan MVC:

- **Model:** Daftar nomor telepon yang tersimpan dalam sistem.
- **View:** Listbox yang menampilkan daftar nomor telepon.
- **Controller:** Logika yang menghubungkan listbox dengan daftar nomor telepon .

Meskipun desain MVC memberikan fleksibilitas yang besar, ia juga memiliki kelemahan, seperti kompleksitas dalam desain awal yang dapat membuat pengembangan lebih sulit jika tidak direncanakan dengan baik.